

Relatório de Atividades: Desenvolvimento de WebService REST com Spring Boot

Disciplina: Sistemas Distribuídos

Instituição: UFC - Campus Quixadá

Autor(a): Gabriely Correia Dealem

Autor(a): Laura Soléria Lobo Vitorino Maciel

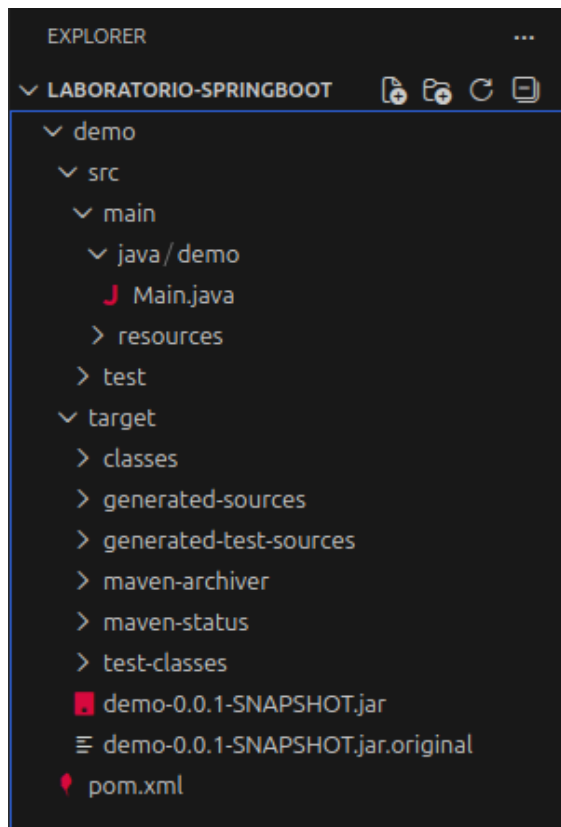
1. Introdução e Objetivo

Este relatório apresenta o desenvolvimento de um componente de software voltado para o estudo de sistemas distribuídos. O objetivo foi criar um **WebService REST** utilizando o framework **Java Spring Boot**, capaz de receber requisições HTTP e processar respostas dinâmicas com base em parâmetros enviados via URL, demonstrando os princípios de comunicação entre processos e transparência de rede.

2. Arquitetura do Sistema

A solução foca na camada de servidor (Back-end) através de uma arquitetura de microsserviço:

- **Servidor:** Uma aplicação Spring Boot que atua como o nó processador.
- **Protocolo:** Comunicação baseada em HTTP/1.1.
- **Formato de Dados:** Texto simples e variáveis de caminho (*Path Variables*).
Shutterstock



3. Implementação do Webservice (Java Spring Boot)

O serviço foi estruturado em uma classe única (**Main.java**), otimizando a inicialização e o roteamento das requisições. As principais tecnologias e anotações utilizadas foram:

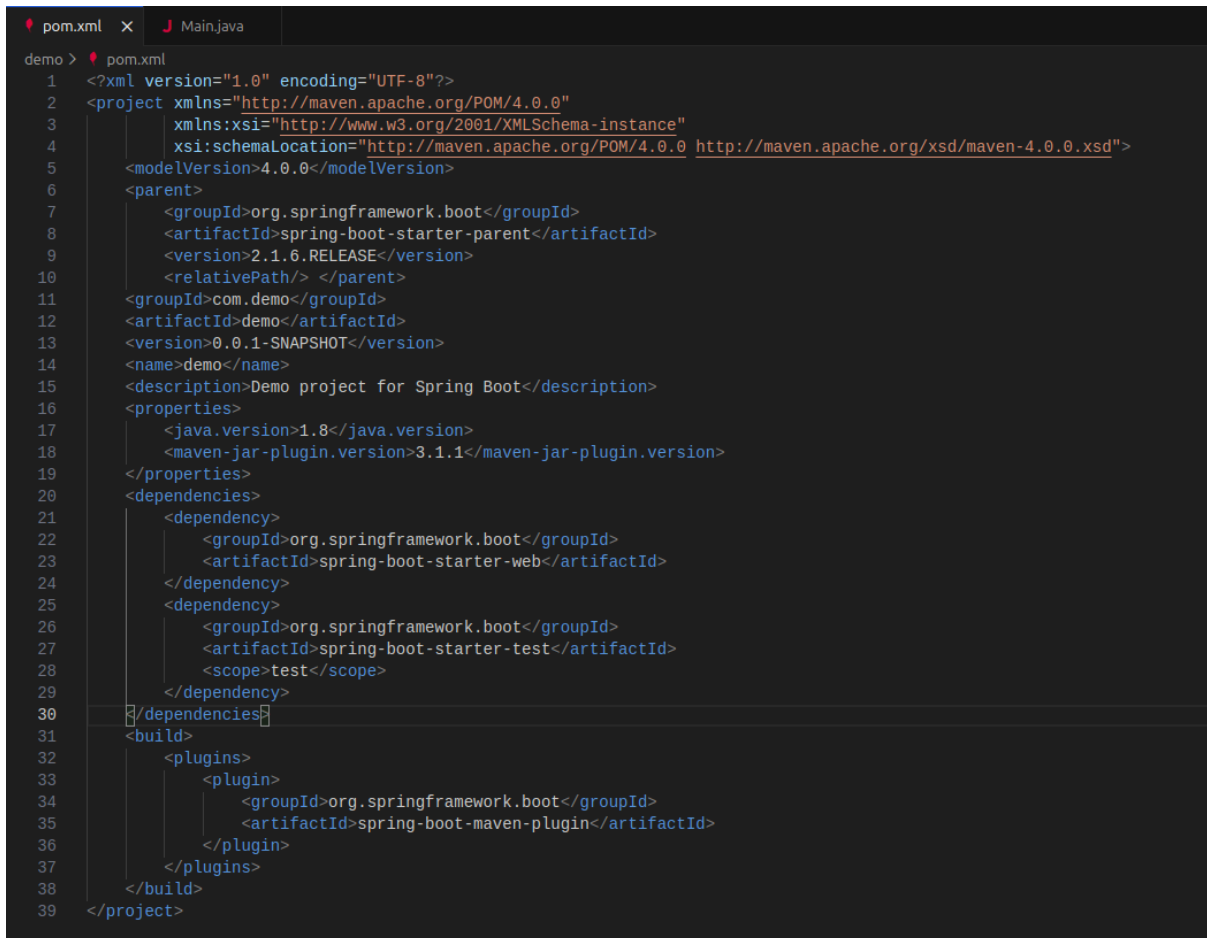
- **@SpringBootApplication**: Marca a classe principal e configura automaticamente o ambiente Spring, incluindo o servidor embutido **Tomcat**.
- **@RestController**: Define que a classe é um controlador REST, onde os métodos retornam dados diretamente no corpo da resposta HTTP.
- **Endpoints**:
 - **/**: Rota raiz que retorna uma mensagem de boas-vindas e identificação da disciplina.
 - **/nome/{nome}**: Rota dinâmica que utiliza **@PathVariable** para capturar o nome do usuário na URL e retornar uma saudação personalizada com uma provocação acadêmica.

```
pom.xml  J Main.java  X
demo > src > main > java > demo > J Main.java
1  package com.demo;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5  import org.springframework.web.bind.annotation.PathVariable;
6  import org.springframework.web.bind.annotation.RequestMapping;
7  import org.springframework.web.bind.annotation.RestController;
8
9  @RestController
10 @SpringBootApplication
11 public class Main {
12
13     @RequestMapping("/")
14     String home() {
15         return "Olá, seja bem vindo ao teste de Spring Boot para disciplina de Sistemas Distribuídos";
16     }
17
18     @RequestMapping("/nome/{nome}")
19     String piada(@PathVariable String nome) {
20         return "Olá " + nome + ", você vai realmente conseguir passar nessa disciplina?";
21     }
22
23     public static void main(String[] args) {
24         SpringApplication.run(Main.class, args);
25     }
26 }
```

4. Gestão de Dependências (Maven)

A configuração do projeto foi realizada através do arquivo `pom.xml`, que gerencia o ciclo de vida da aplicação:

- **Spring Boot Starter Web:** Dependência fundamental que fornece as bibliotecas necessárias para criar aplicações web, incluindo suporte a REST e validação.
- **Java 1.8:** Definido como a versão de compatibilidade para o compilador.
- **Spring Boot Maven Plugin:** Responsável por empacotar a aplicação em um arquivo JAR executável, facilitando a portabilidade do sistema.



```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6     <parent>
7         <groupId>org.springframework.boot</groupId>
8         <artifactId>spring-boot-starter-parent</artifactId>
9         <version>2.1.6.RELEASE</version>
10        <relativePath/> </parent>
11    <groupId>com.demo</groupId>
12    <artifactId>demo</artifactId>
13    <version>0.0.1-SNAPSHOT</version>
14    <name>demo</name>
15    <description>Demo project for Spring Boot</description>
16    <properties>
17        <java.version>1.8</java.version>
18        <maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
19    </properties>
20    <dependencies>
21        <dependency>
22            <groupId>org.springframework.boot</groupId>
23            <artifactId>spring-boot-starter-web</artifactId>
24        </dependency>
25        <dependency>
26            <groupId>org.springframework.boot</groupId>
27            <artifactId>spring-boot-starter-test</artifactId>
28            <scope>test</scope>
29        </dependency>
30    </dependencies>
31    <build>
32        <plugins>
33            <plugin>
34                <groupId>org.springframework.boot</groupId>
35                <artifactId>spring-boot-maven-plugin</artifactId>
36            </plugin>
37        </plugins>
38    </build>
39 </project>

```

5. Procedimentos de Execução

5.1. Ajuste de Ambiente (Compatibilidade Java 8)

Para garantir que o projeto execute sem conflitos de versão, especialmente em sistemas com JDKs mais recentes, devem ser seguidos os seguintes comandos no terminal:

1. **Instalação do JDK compatível:**
 - `sudo apt update`
 - `sudo apt install openjdk-8-jdk`
2. **Configuração temporária das variáveis de ambiente:**
 - `export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64`
 - `export PATH=$JAVA_HOME/bin:$PATH`

5.2. Inicialização do Servidor

Dentro da pasta raiz do projeto (diretório `demo`), execute o comando para compilar e iniciar o serviço:

- `mvn clean spring-boot:run`

O servidor é iniciado por padrão na porta **8080**.

5.3. Teste de Conectividade

A validação foi feita via navegador ou ferramentas de teste de API (como Postman/Insomnia) acessando as URLs:

- <http://localhost:8080/>
- <http://localhost:8080/nome/SeuNome>

6. Desafios e Resolução de Problemas

Durante a configuração e execução, foram enfrentados desafios técnicos relevantes:

- **Localização do Projeto:** O Maven exige a presença do arquivo `pom.xml` no diretório de execução. Foi necessário navegar até a subpasta correta para o reconhecimento do projeto.
- **Incompatibilidade de Versões:** O uso da versão `2.1.6.RELEASE` do Spring Boot com o Java 17+ gerou o erro `NoClassDefFoundError: MBeanFactory`. A solução foi o *downgrade* controlado para o Java 8 no terminal, conforme descrito na seção anterior.

7. Validação dos Resultados

O sistema respondeu conforme o esperado para as entradas fornecidas:

- **Entrada:** Acesso à URL <http://localhost:8080/nome/Joao>
- **Processamento:** O Spring Boot mapeou o valor "Joao" para a variável `nome`.
- **Saída:** "Olá Joao, você vai realmente conseguir passar nessa disciplina?"

8. Conclusão

A atividade permitiu compreender na prática como um serviço distribuído disponibiliza recursos para outros nós da rede. A facilidade de configuração do Spring Boot demonstra a eficiência do paradigma de "convenção sobre configuração", permitindo que o desenvolvedor foque na lógica do sistema distribuído em vez de configurações complexas de infraestrutura de servidor.